

Absolutely. Since you're starting HLD as a **beginner**, I'll teach these from the ground up rather than assuming you already know distributed systems.

The goal is not just to memorize definitions. In an HLD interview, you want to develop a mental model that lets you answer:

“If millions of users use this system, how would you design it?”

We'll build that ability step by step.

1. HLD Interview Approach at Top Product-Based Companies

First, understand what an HLD interview actually tests.

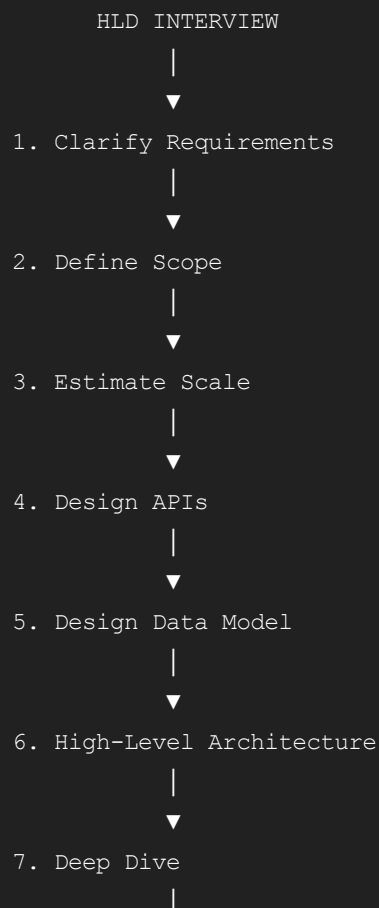
It is **not primarily a coding interview**.

The interviewer wants to see whether you can take something vague like:

“Design YouTube.”

and turn it into a structured engineering problem.

A good HLD discussion usually follows this flow:



Let's understand each.

Step 1: Clarify requirements

Suppose interviewer says:

Design Instagram.

Don't immediately start drawing:

Client → Load Balancer → Server → Database

That's a common beginner mistake.

First ask questions.

For example:

Functional requirements

- Can users upload photos?
- Can users follow other users?
- Can users like/comment?
- Should users see a feed?
- Can users send messages?

Non-functional requirements

- How many users?
- How much traffic?
- What latency do we need?
- Is high availability important?
- How much data are we storing?

You don't need to ask 30 questions.

Usually ask enough questions to establish the **scope of your design**.

Step 2: Define scope

You should explicitly say:

"For this design, I'll focus on photo upload, follow/unfollow, and the home feed. I'll leave messaging and stories out of scope."

This is important.

Otherwise the interview can become infinite.

Step 3: Estimate scale

Now ask:

How big is this system?

Suppose:

```
100 million users
10 million daily active users
1 million requests/sec
```

These numbers affect your architecture.

A system handling:

```
100 requests/sec
```

can be very different from:

```
1,000,000 requests/sec
```

We'll study estimation in detail below.

Step 4: Design APIs

Before thinking about databases, think:

What operations does the system expose?

For Instagram:

```
http

POST /users/{userId}/posts

GET /users/{userId}/feed
```

```
POST /posts/{postId}/like
```

```
POST /users/{userId}/follow
```

Then define request/response structures.

Step 5: Data modelling

Now ask:

What information do we need to store?

For example:

```
User
-----
user_id
name
email
created_at
```

```
Post
-----
post_id
user_id
image_url
caption
created_at
```

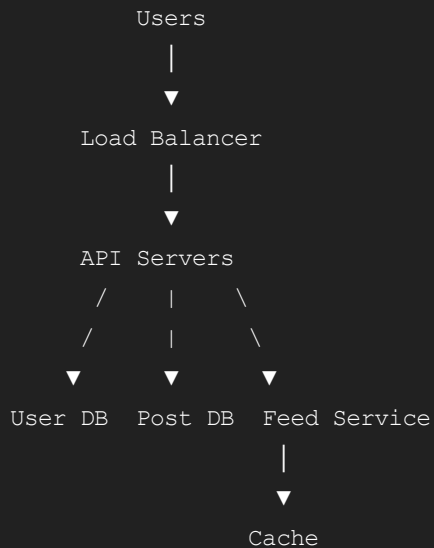
```
Follow
-----
follower_id
following_id
created_at
```

Then decide whether SQL or NoSQL makes more sense.

Step 6: High-level architecture

Only now start drawing architecture.

For example:



Then add things like:

CDN
Message Queue
Object Storage
Search
Database Replicas

as the requirements demand.

Step 7: Deep dive

The interviewer might say:

"Your feed service seems interesting. How does it work?"

Now you go deeper.

For example:

User follows 10,000 people.

How do we generate their feed?

You might discuss:

- Fan-out on write
- Fan-out on read
- caching
- celebrity users
- ranking

- queues

This is where strong HLD interviews often differentiate candidates.

Step 8: Trade-offs

This is **extremely important**.

There is rarely one perfect architecture.

For example:

| Should we use SQL or NoSQL?

Wrong mindset:

| "MongoDB is better."

Better mindset:

| "Given these access patterns and consistency requirements, I would choose X because..., but it has these trade-offs."

HLD is largely about **trade-offs**.

The mental model you should develop

Whenever you see a system-design question, think:

```
Requirements
  ↓
Scale
  ↓
APIs
  ↓
Data
  ↓
Architecture
  ↓
Bottlenecks
  ↓
Trade-offs
```

Keep this sequence in your head.

2. CAP Theorem

This is one of the most important distributed-systems concepts.

Imagine you have a database with two servers:

```
Database
 /      \
A        B
```

A user writes data to A.

Normally A and B communicate.

But suddenly:

```
Database
 /      \
A  X    B
 \      /
Network
broken
```

A and B can no longer communicate.

This is called a **network partition**.

Now the system has a problem.

Suppose:

```
A says:
```

```
User balance = ₹100
```

but B still says:

```
User balance = ₹50
```

What should happen if users send requests to both?

That's where CAP comes in.

CAP = Consistency + Availability + Partition Tolerance

C — Consistency

Every read gets the **latest valid value**.

Example:

```
Write:  
balance = ₹100  
  
Immediately read:  
balance = ₹100
```

You don't get an old value.

A — Availability

Every request receives a response, even if some nodes are unavailable.

Example:

```
Server A is down  
  
Request → Server B  
  
Server B still responds.
```

P — Partition Tolerance

The system continues operating despite communication failures between nodes.

Example:

```
Node A ↔ Node B
```

The network connection is broken.

A partition has occurred.

A partition-tolerant system continues functioning.

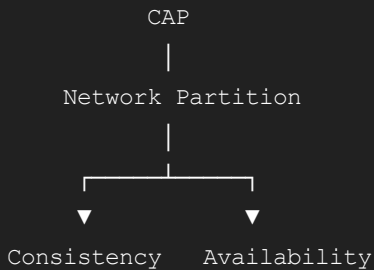
The famous CAP rule

When a network partition occurs, you generally have to choose between:

Consistency
OR
Availability

You cannot guarantee both simultaneously during the partition.

So:



And because distributed systems must generally tolerate network failures:

P is usually treated as non-negotiable.

So practical systems often make a trade-off between:

CP

and

AP

CP system

Consistency + Partition tolerance

During a partition, the system may reject/delay some requests rather than return potentially stale data.

Example conceptual scenario:

Node A $\leftrightarrow$ Node B

Request arrives.

System says:

"Can't guarantee correct latest data."

Availability decreases, but consistency is preserved.

Useful when incorrect data is dangerous.

Examples can include certain:

- financial operations
- distributed coordination systems
- strongly consistent metadata

AP system

Availability + Partition tolerance

During a partition, the system continues responding, potentially returning stale data.

Example:

```
Node A ↔ Node B
```

```
User requests data
```

```
→ return locally available data
```

The data may become consistent later.

This is often appropriate for systems where temporary staleness is acceptable.

For example:

```
Social media likes
```

```
View counters
```

```
Some recommendation data
```

If someone sees:

```
Likes: 1,203
```

instead of:

```
Likes: 1,204
```

for a short period, that's generally less problematic than refusing the request entirely.

Very important CAP interview nuance

Don't say:

"MongoDB is AP and PostgreSQL is CA."

That's an oversimplification.

CAP describes **distributed systems under network partition**, and the behavior depends on the actual architecture, configuration, replication model, and operation being discussed.

The important interview concept is:

During a network partition, do we prioritize consistency or availability?

3. Consistency Models

CAP tells us there is a trade-off.

Consistency models tell us **how fresh/ordered data needs to appear to users**.

The three you mentioned are particularly important.

Eventual Consistency

Imagine:



User writes:

```
name = "Saurabh"
```

Database A immediately knows.

Database B might temporarily still have:

```
name = "Rahul"
```

After replication:

```
Database B  
↓  
name = "Saurabh"
```

That's **eventual consistency**.

The system guarantees that if updates stop, replicas will eventually converge.

Advantage

Very scalable and highly available.

Disadvantage

Users may temporarily see stale data.

Strong Consistency

A read after a successful write is expected to see the latest committed value according to the system's consistency guarantees.

Example:

```
WRITE  
balance = ₹100  
↓  
successful  
  
READ  
↓  
₹100
```

You don't want:

```
READ → ₹50
```

after the successful update if your system promises strong consistency for that operation.

Useful for things such as:

- account balances
- inventory
- critical configuration
- certain transactional operations

Linearizability

Linearizability is stronger and more precise.

Think of every operation as if it happened at **one exact point in time** between when the operation started and when it completed.

Suppose:

```
10:00:00 → Write X = 10
10:00:01 → Write completes
10:00:02 → Read X
```

A linearizable system must return:

```
X = 10
```

because the read happened after the completed write.

The important idea:

Operations appear to happen atomically in real-time order.

Strong consistency vs linearizability

For interviews, don't casually treat every use of "strong consistency" as mathematically identical to linearizability.

A useful mental model is:

```
Eventual consistency
↓
```

more relaxed

Strong consistency

↓

more restrictive

Linearizability

↓

a specific strong real-time ordering guarantee

In practical HLD discussions, interviewers may use "strongly consistent" loosely, so clarify what guarantee you're assuming.

Easy example

Imagine changing your username.

Eventual consistency

Device A → new username

Device B → old username

after a few seconds

Device B → new username

Strong/linearizable behavior

Update succeeds

↓

subsequent read

↓

new username

4. SQL vs NoSQL

This is another classic interview question.

First:

SQL doesn't mean old.

NoSQL doesn't mean automatically scalable.

The decision depends primarily on:

Data model + access patterns + consistency + scale + operational requirements.

SQL

Examples:

- PostgreSQL
- MySQL
- Oracle
- SQL Server

SQL databases generally organize data into tables.

Example:

Users

id	name	email
1	Saurabh	...
2	Rahul	...

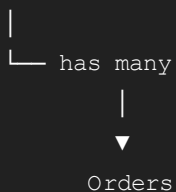
Another table:

Orders

id	user_id	amount
1	1	500
2	1	800

Relationship:

User



Why SQL is powerful

1. Relationships

Suppose:

```
Customer
  ↓
Orders
  ↓
Products
```

SQL handles relationships naturally.

2. Transactions

Suppose transferring ₹500:

```
Account A: -₹500
Account B: +₹500
```

You don't want:

```
A updated
B failed
```

A transaction can ensure the operation behaves atomically according to the transaction's guarantees.

3. Strong consistency

Many SQL systems provide strong transactional consistency within a database.

4. Powerful querying

You can ask:

SQL

```
SELECT *
FROM orders
WHERE user_id = 123
AND amount > 1000;
```

You can also use:

- joins
 - aggregations
 - sorting
 - filtering
 - constraints
-

NoSQL

NoSQL is a broad category rather than one single database design.

Examples include:

Document DB

MongoDB

JSON

```
{
  "userId": 123,
  "name": "Saurabh",
  "skills": ["Java", "Kotlin", "Python"]
}
```

Key-value

Redis

```
"user:123" → "{user data}"
```

Wide-column

Cassandra

```
partition key → rows
```

Graph

Neo4j

```
Saurabh
  ↓ follows
Rahul
  ↓ follows
Amit
```

Different NoSQL databases solve different problems.

When would you consider NoSQL?

Suppose you have:

```
1 billion users
```

and your primary access pattern is:

```
GET user by userId
```

You may want a database optimized for:

```
key → value
```

rather than complicated joins.

NoSQL can be attractive when you need:

- massive horizontal scaling
- flexible schema
- high write throughput
- predictable key-based access
- distributed storage

But again:

NoSQL ≠ automatically better.

SQL vs NoSQL mental model

Think:

Requirement	Often points toward
Complex relationships	SQL
Transactions	SQL
Complex queries	SQL
Strong relational integrity	SQL

Requirement	Often points toward
Flexible schema	NoSQL
Huge distributed scale	NoSQL may fit
Key-value access	Key-value NoSQL
Massive write throughput	Some NoSQL systems
Graph relationships	Graph DB

These are tendencies, not absolute rules.

5. Back-of-the-Envelope Estimation

This is one of the most useful HLD skills.

The interviewer might ask:

"How much traffic will your system receive?"

You don't need the exact answer.

You need to show that you can make a **reasonable estimate**.

The most important numbers

Usually estimate:

Users
DAU
Requests/day
QPS
Peak QPS
Storage
Bandwidth

Example: URL Shortener

Suppose:

100 million users

But let's say:

10 million URLs created/day

We want average write QPS.

There are:

86,400 seconds/day

Therefore:

$$\begin{aligned} &10,000,000 / 86,400 \\ &\approx 116 \text{ QPS} \end{aligned}$$

So approximately:

116 writes/sec

Peak QPS

Real systems aren't uniformly loaded.

Suppose peak traffic is approximately 5× average.

Then:

$$\begin{aligned} &116 \times 5 \\ &\approx 580 \text{ QPS} \end{aligned}$$

So you might design for roughly:

~600 write QPS

The exact multiplier isn't universal; state your assumption.

Read QPS

Suppose each URL is accessed 100 times on average.

Then:

```
10M URLs × 100  
= 1 billion reads/day
```

QPS:

```
1,000,000,000 / 86,400  
≈ 11,574 QPS
```

Peak:

```
≈ 58,000 QPS
```

Now you have a very different architecture.

The system is:

```
Write-heavy?  No  
Read-heavy?   YES
```

That immediately suggests thinking about:

- caching
- replicas
- CDN
- partitioning

Storage estimation

Suppose each URL record requires roughly:

```
500 bytes
```

And we create:

```
10 million/day
```

Storage/day:

```
10M × 500 bytes  
= 5 GB/day
```

One year:

```
5 × 365  
≈ 1.8 TB/year
```

Then account for:

```
indexes  
replication  
metadata  
overhead
```

Maybe your actual requirement becomes several TB.

You don't need fake precision.

Say:

"Roughly 2 TB/year before replication and index overhead."

That's good HLD communication.

Bandwidth estimation

Suppose:

```
100,000 requests/sec
```

and average response size:

```
10 KB
```

Bandwidth:

```
100,000 × 10 KB  
= 1,000,000 KB/sec  
≈ 1 GB/sec
```

This is significant.

Now you start asking:

Can a single server handle this?

Probably not.

Then you think:

```
Load Balancer
  ↓
Multiple servers
```

and possibly:

```
CDN
```

for cacheable content.

Memorize these

```
1 day = 86,400 seconds
1 month ≈ 2.6 million seconds
1 year ≈ 31.5 million seconds
```

And:

```
QPS = requests/day ÷ 86,400
```

Peak:

```
Peak QPS = Average QPS × peak factor
```

Storage:

```
Storage = number of objects × average object size
```

Bandwidth:

```
Bandwidth = requests/sec × average request/response size
```

These simple formulas will take you surprisingly far.

6. API Design

Now let's talk APIs.

Suppose we're designing a URL shortener.

We need:

```
POST /urls
```

Create a short URL.

Request:

JSON

```
{  
  "longUrl": "https://example.com/very/long/url"  
}
```

Response:

JSON

```
{  
  "shortCode": "aB92x",  
  "shortUrl": "https://sho.rt/aB92x"  
}
```

Then:

```
GET /{shortCode}
```

returns a redirect.

Good API design considers

1. Resource

What are we operating on?

```
/users  
/orders  
/products  
/posts
```

2. Operation

GET
POST
PUT
PATCH
DELETE

3. Request

What information does the server need?

4. Response

What does the client receive?

5. Error handling

For example:

```
404 → resource doesn't exist
400 → invalid request
401 → unauthenticated
403 → not authorized
429 → too many requests
500 → server error
```

Idempotency

Very important in HLD.

Suppose:

```
http
POST /payments
```

Client sends payment request.

Network fails.

Client doesn't know whether payment succeeded.

It retries.

Now potentially:

```
₹1000
```

gets charged twice.

To solve this, use an **idempotency key**.

```
http
```

```
POST /payments
```

```
Idempotency-Key: abc123
```

Server remembers:

```
abc123 → payment result
```

If the same request comes again:

```
abc123
```

return the previous result instead of creating another payment.

This is a very useful concept to know for interviews.

Pagination

Suppose:

```
http
```

```
GET /users/123/posts
```

User has:

```
1,000,000 posts
```

Don't return all of them.

Use pagination.

For example:

```
http
```

```
GET /users/123/posts?limit=20&cursor=abc
```

Cursor-based pagination is often preferable for large/changing datasets.

7. Data Modelling

This is where you translate the requirements into stored entities.

Let's design a simple Instagram-like system.

Entities:

```
User
Post
Comment
Like
Follow
```

User

```
User
-----
user_id
name
email
created_at
```

Post

```
Post
-----
post_id
user_id
caption
media_url
created_at
```

Relationship:

```
User 1 ————— N Post
```

One user can create many posts.

Comment

```
Comment
-----
comment_id
post_id
user_id
text
created_at
```

Relationship:

Post 1 ——— N Comment

Like

```
Like
-----
user_id
post_id
created_at
```

You might create a uniqueness constraint:

(user_id, post_id)

so the same user cannot like the same post twice.

Follow

This is interesting.

```
Follow
-----
follower_id
following_id
created_at
```

Example:

Saurabh → follows → Rahul

becomes:

```
follower_id = Saurabh  
following_id = Rahul
```

This is effectively a relationship table.

Data modelling starts from access patterns

This is **one of the biggest HLD lessons**.

Don't only ask:

"What data do I have?"

Ask:

"How will I access this data?"

For example:

```
GET all posts from user 123
```

suggests an index around:

```
user_id + created_at
```

If your main query is:

```
GET followers of user 123
```

you want efficient lookup by:

```
following_id
```

So:

Data model + indexes should be designed around access patterns.

A complete beginner mental model

Put everything together.

Suppose interviewer says:

Design a food-delivery application.

Your thought process should become:

1. Requirements

```
Users
Restaurants
Menus
Orders
Payments
Delivery tracking
```

2. Scale

```
10M DAU
500K orders/day
Peak QPS ≈ ...
Storage ≈ ...
```

3. APIs

```
GET /restaurants
GET /restaurants/{id}/menu
POST /orders
GET /orders/{id}
POST /payments
```

4. Data

```
User
Restaurant
Menu
Order
OrderItem
Payment
Delivery
```

5. Database

Perhaps:

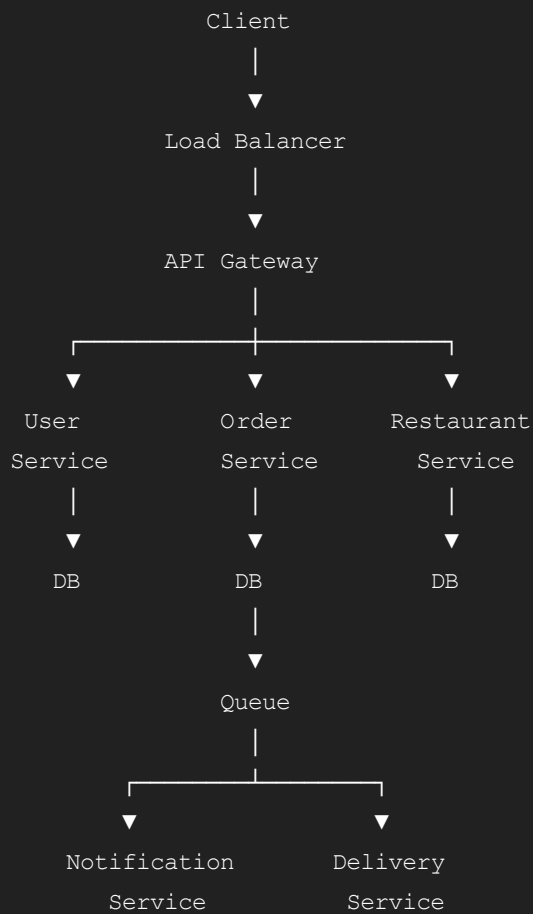
```
SQL
→ orders
```

→ payments
→ transactional data

Potentially:

NoSQL / cache
→ high-scale read paths
→ location/real-time use cases depending on requirements

6. Architecture



7. Deep dive

Then interviewer chooses something:

"How would you handle 100,000 orders/minute?"

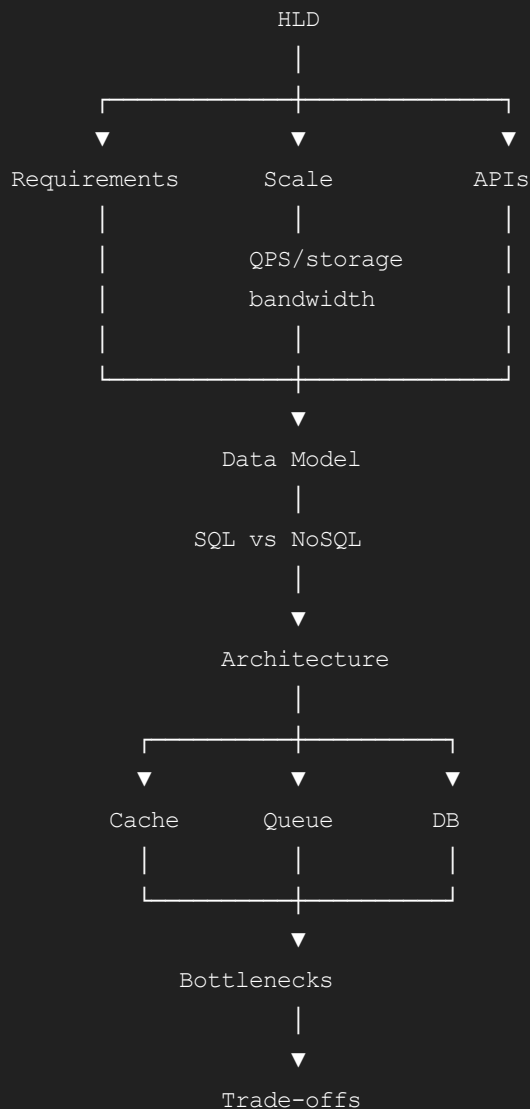
Now you discuss:

- partitioning
- queues
- caching
- database scaling
- replicas

- idempotency
- failure handling

Your HLD Foundation Cheat Sheet

I'd memorize this mental map:



And these are the **5 things** I'd want you to become comfortable with first:

1. **Requirements** → **scope**
2. **QPS/storage/bandwidth** estimation
3. **SQL vs NoSQL** based on access patterns
4. **Consistency + CAP** trade-offs
5. **APIs + data** modelling

Once these are comfortable, things like **load balancers, caching, sharding, replication, message queues, CDNs, Kafka, Redis, microservices, etc.** become much easier to

understand because you'll know *why* they're being introduced rather than memorizing architecture diagrams.

A useful interview habit

When designing anything, repeatedly ask yourself:

"What problem am I solving by adding this component?"

For example:

```
Redis
  ↓
Why?
  ↓
Reduce database reads / latency

Kafka
  ↓
Why?
  ↓
Decouple services / absorb bursts / asynchronous processing

Load Balancer
  ↓
Why?
  ↓
Distribute traffic + improve availability

Database Replica
  ↓
Why?
  ↓
Scale reads / improve availability

Sharding
  ↓
Why?
  ↓
Database is too large or too much traffic for one node
```

That **problem** → **solution** → **trade-off** way of thinking is much more valuable in HLD interviews than memorizing a list of technologies.

Next, I'd recommend going **much deeper into CAP + consistency models with real interview examples**, because that is usually where beginners start getting confused.